
Sprawozdanie z listy 2

Laboratorium ze sztucznej inteligencji - implementacja
minimax

Karol Wołkowski

2026-04-11

Spis treści

1	Sprawozdanie z listy 2	2
1.1	Wybór języka	2
1.2	Założenia gry	2
1.3	Modelowanie gry	2
1.4	Heurestyki	4
1.4.1	PawnsAdvantageHeuristic (Przewaga materiałowa) - Simple	4
1.4.2	ProgressiveHeuristic (Agresywny postęp)	4
1.4.3	BalancedLeadHeuristic (Zbalansowany Lider)	5
1.5	Minimax	5
1.6	Optymalizacja Alfa Beta Cięcie	7
1.7	Ocena heurestyk	8
1.7.1	Średnia liczba rund dla każdej z heurestyk	9
1.7.2	Liczba odwiedzonych węzłów	9
1.7.3	Średnia wartość zwycięstwa	10

1 Sprawozdanie z listy 2

1.1 Wybór języka

Jako język do napisania implementacji wybrałem *Scala*. Problem dobrze nadaje się do modelowania funkcyjnego, ponieważ generowanie stanów, ewaluacja pozycji oraz rekurencyjne przeszukiwanie drzewa gry można wyrazić za pomocą stosunkowo prostych funkcji. Sam paradygmat funkcyjny stanowił jednak pewne wyzwanie, ponieważ na co dzień programuję głównie w stylu imperatywnym.

Wykorzystane biblioteki to:

- biblioteka standardowa scali i javy
- `scopt` do budowy cli (niezwiązane z algorytmem)

Wersja języka to scala 3.x

1.2 Założenia gry

W klasycznym ujęciu algorytmu minimax przyjmujemy, że gracz maksymalizujący (np. biały) wybiera ruchy prowadzące do możliwie największej wartości funkcji oceny, natomiast gracz minimalizujący (np. czarny) wybiera ruchy prowadzące do wartości możliwie najmniejszej.

W mojej implementacji końcowo zastosowany został jednak wariant Negamax, w którym oba przypadki sprowadza się do jednej operacji maksymalizacji, a zmiana perspektywy gracza jest uwzględniana przez negację wyniku.

Zasady gry breakthrough polegają na:

- zdobyciu wiersza najbliższego przeciwnikowi
- wybiciu wszystkich pionków przeciwnika

Co najważniejsze, w grze niemożliwy jest remis, ponieważ poruszać się możemy tylko do przodu lub być bitym.

1.3 Modelowanie gry

Do zamodelowania stanu gry przyjąłem metodę kodowania koordynatów. Dla każdej z pozycji na planszy przyjąłem oznaczenie inspirowane oznaczeniami z szachów. Dlatego numerowanie rozpoczynamy dla gracza z pionkami białymi, poruszając się w prawo, i przeskakując na początek kolejnego wiersza gdy zbliżymy się do końca obecnego.

Przykładowa plansza 8x8:

1	B	B	B	B	B	B	B	B	57	58	59	60	61	62	63	64
2	B	B	B	B	B	B	B	B	49	50	51	52	53	54	55	56
3	-	-	-	-	-	-	-	-	41	42	43	44	45	46	47	48
4	-	-	-	-	-	-	-	-	33	34	35	36	37	38	39	40
5	-	-	-	-	-	-	-	-	25	26	27	28	29	30	31	32
6	-	-	-	-	-	-	-	-	17	18	19	20	21	22	23	24
7	W	W	W	W	W	W	W	W	9	10	11	12	13	14	15	16
8	W	W	W	W	W	W	W	W	1	2	3	4	5	6	7	8

W takim modelu, możemy w bardzo łatwy sposób matematycznie obliczać ruchy, bicia, i generować kolejne ruchy. Plansza zaczyna numerowanie zawsze z perspektywy gracza pierwszego. W powyższym przykładzie pozycje startowe to:

- dla gracza pierwszego **od 1 do 16**
- dla gracza drugiego **od 49 do 64**

Aby poprawnie obliczać ruchy, należy wziąć pod uwagę perspektywę każdego z graczy. Można zauważyć że gracz pierwszy porusza się zawsze dodając wartości do swojej pozycji, natomiast gracz drugi - odejmując. W ogólnym założeniu możemy dokładnie sklasyfikować 3 przesunięcia:

- offset: n - 1** - gracz pierwszy porusza się w lewy skos, natomiast drugi w prawy skos
- offset: n** - gracz pierwszy i drugi poruszają
- offset: n + 1** - gracz pierwszy porusza się w prawy skos, natomiast drugi w lewy skos

gdzie n to rozmiar planszy $n \times n$

Należy również wziąć pod uwagę fakt że powyższa reguła nie jest spełniona jeżeli:

- na drodze stoi nam pionek (w lini prostej)
- znajdujemy się na którymś ze skrajów planszy

Drugi przypadek da się łatwo rozwiązać matematycznie za pomocą prostych równań. Przy ruchu **w lewy skos** (offset n - 1): `target % boardSize` nie może wyjść 0 (czyli pionek nie może wylądować w 8. kolumnie, co oznaczałoby, że "przeskoczył" z lewej krawędzi na prawą krawędź poprzedniego rzędu). Przy ruchu w prawy skos (offset n+1): `target % boardSize` nie może wyjść 1 (pionek nie może wylądować w 1. kolumnie, przeskakując z prawej krawędzi na lewą). Dla gracza 2 logika jest lustrzanym odbiciem.

Aby łatwo ograniczać ruch każdego z graczy tylko do swoich pionków, i zamodelować kolizje (brak możliwości ruchu przez blokadę innego pionka) **stan gry jest podzielony na 2 zbiory liczb reprezentujące pionki każdego gracza

1.4 Heurystyki

Aby odpowiednio ewaluować stany gry, dla funkcji minimax, przygotowałem następujące propozycje heurystyk oceny stanu gry. Wszystkie heurystyki oceniają stan gry z perspektywy gracza mającego turę.

1.4.1 PawnsAdvantageHeuristic (Przewaga materiałowa) - Simple

Jest to najprostsza zaimplementowana heurystyka. Skupia się wyłącznie na liczbie pionów na planszy, ignorując ich pozycję.

- Logika: Oblicza różnicę między liczbą własnych pionów a liczbą pionów przeciwnika.
- Wzór: $score = N_{gracz} - N_{przeciwnik}$

Aby delikatnie naprowadzić heurystykę, która może zbliżyć się już do wygranej, zastosowałem bardzo dużą wartość (w porównaniu do normalnych wartości dla tej heurystyki) kiedy gra jest skończona z wygraną dla gracza, oraz bardzo niską wartość kiedy jest przegrana dla gracza. Ma to na celu delikatne naprowadzenie w końcowych fazach algorytmu minimax aby program zbiegał do wygranej.

1.4.2 ProgressiveHeuristic (Agresywny postęp)

Heurystyka ta nadaje priorytet pionom znajdującym się najbliżej linii mety przeciwnika, stosując wzrost wykładniczy.

- Logika: Każdy pion jest wart $2^{progress}$ punktów, gdzie progress to numer rzędu, w którym znajduje się pion (licząc od 0 z perspektywy gracza).
- Wzór: $score = \sum_{p \in pozycje} 2^{progress(p)}$

1.4.3 BalancedLeadHeuristic (Zbalansowany Lider)

Heurystyka stanowi kompromis między utrzymaniem dużej liczby pionów a promowaniem liderów. Używa postępu liniowego zamiast wykładniczego, co zapobiega zbyt wczesnym biciom przez przeciwnika.

- Logika:

1. Każdy pion ma bazową wartość (10 pkt) powiększoną o bonus za każdy rząd postępu (2 pkt).
2. Dodatkowo przyznawany jest bonus za najbardziej wysuniętego piona (lidera), co zachęca do wygrywania, ale nie kosztem utraty wszystkich innych figur.

- Wzór: $score = \sum_{p \in \text{pozycje}} (10 + 2 \cdot \text{progress}(p)) + 5 \cdot \max(scores_{\text{pion}})$

- gdzie $scores$ to zbiór jednostkowych ocen, przed sumowaniem

1.5 Minimax

Do implementacji posłużyłem się 2 źródłami. Pierwszym z nich była implementacja algorytmu minimax w grze “kółko i krzyżyk”, również napisana w scali. Jej autorem jest *Yurii Lahodiuk*. Implementacja jest najprostszą wersją algorytmu, tzn. nie posiada optymalizacji takich jak ewaluacja do pewnej głębokości czy choćby cięcia alfa beta. Z powodu braku wczesnej ewaluacji, implementacja pozbawiona jest również heurystyk; stan gry oceniany jest tylko przez pryzmat wygrana/przegrana

Drugim źródłem była praca magisterska Piotra Belinga z Politechniki Łódzkiej z roku 2006. Jej tytuł brzmi “*Praktyczne aspekty programowania gier logicznych*” i jest przejściem przez wiele różnych implementacji algorytmów z teorii gier. Bardzo interesująca okazała się dla mnie wzmianka o wariantach algorytmu *minimax*, będącym matematycznym uproszczeniem algorytmu oryginalnego, ale w istocie działającym na takiej samej zasadzie. Mowa o algorytmie NegaMax, który w przeciwieństwie do minimax, maksymalizuje wartość dla każdego z graczy

Kluczową cechą mojej implementacji było to, że zarówno heurystyki, jak i sam stan gry były definiowane z perspektywy gracza mającego ruch. Taka konwencja utrudniałaby bezpośrednie zastosowanie klasycznej wersji minimaksu, ale bardzo dobrze współgra z algorytmem Negamax.

W pracy możemy przeczytać o ciekawej zależności:

$$\forall a_1, a_2, \dots, a_N \in \mathbb{R} : \min(a_1, a_2, \dots, a_N) = -\max(-a_1, -a_2, \dots, -a_N)$$

Dzięki temu możemy całkowicie ominąć minimalizację i zająć się maksymalizacją wyniku dla każdego z graczy, negując wartości w odpowiednim momencie. Możemy zauważyć, że ocena stanu gry dla każ-

dego z dzieci obecnego stanu, jest już oceną z perspektywy gracza drugiego czyli przeciwnika (ponieważ nowy stan ma zmienioną turę). Z tego powodu wartość jaką zwraca taka ewaluacja jest wartością “niekorzystną” dla nas, Dlatego wynik zwrócony przez rekurencyjne wywołanie musi zostać zanegowany, aby ponownie wyrażał ocenę z perspektywy bieżącego gracza. Dzięki temu w każdym węźle możemy szukać po prostu maksimum.

Oto przykładowy pseudokod dla algorytmu, zwanym dalej **Nega-Max** jako alternatywa/rozszerzenie algorytmu minimax

```
1 int NegaMax(Stan s, int Depth) {
2     if (Depth == 0)
3         return ocena(S)
4     vector<Stan> N = nast(S);
5     if (N == empty)
6         return wypłata(S);
7     int result = -Infinity;
8     for (int i = 0; i < |N|; i++){
9         int val = -NegaMax(N[i], Depth - 1);
10        if (val > result)
11            result = val;
12    }
13    return result
14 }
```

Powyższy kod da się bardzo łatwo zapisać w formie funkcyjnej.

```
1 def negamax(s: S, depth: Int): Double =
2     if depth == 0 || s.isGameOver then heuristic(s)
3     else
4         s.generateStates.foldLeft(Double.NegativeInfinity)((max, state) =>
5             {
6                 val score = -1 * negamax(state, depth - 1)
7                 if score > max then score
8                 else max
9             })
10 // S to śjaka implementacja stanu, która implementuje metod `
11 //   generateStates`,
12 //   a zwracając wszystkie możliwe stany gry
```

Important

W tym miejscu należy podkreślić że moja początkowa konkretyzacja heurystyki, polegająca na ewaluacji stanu gry **względem gracza mającego turę** jest w tym przypadku wręcz obowiązkowa.

1.6 Optymalizacja Alfa Beta Cięcie

Aby przygotować tę optymalizację dla algorytmu *NegaMax*, należy trochę zmienić myślenie o tych dwóch parametrach. W przytoczonej wcześniej pracy magisterskiej możemy przeczytać że:

Przekazując α i β do wywołań rekurencyjnych (linia 13) trzeba pamiętać, iż interpretacja tych liczb wewnątrz tego wywołania następuje z punktu widzenia rywala G, dlatego też należy te wartości zanegować i zamienić miejscami

```
1 // pseudokod z przytoczonej pracy
2 int AlfaBeta(Stan s, int Depth, int a, int b) {
3     if (Depth == 0)
4         return ocena(S)
5     vector<Stan> N = nast(S);
6     if (N == empty)
7         return wypłata(S);
8     int result = -Infinity;
9     for (int i = 0; i < |N|; i++){
10         int val = -AlfaBeta(N[i], Depth - 1, -b, -a);
11         if (val >= b)
12             return b;
13         if (val > a)
14             a = val;
15     }
16     return a
17 }
```

Początkowo taka implementacja, a w szczególności zamiana parametrów α i β wraz z negacją mogą wydawać się niejasne. Aby to sobie wyjaśnić trzeba spojrzeć na to, czym tak naprawdę są te 2 wartości:

- α - najlepsza wartość znaleziona dotąd dla bieżącego gracza, czyli dolne ograniczenie wartości aktualnego wężła
- β - górne ograniczenie wartości aktualnego wężła wynikające z wcześniejszych decyzji przeciwnika; po jego osiągnięciu dalsze przeszukiwanie nie ma sensu.

widzimy że zachodzi nierówność:

$$\alpha \leq v \leq \beta$$

Dla drugiego gracza ta sama pozycja ma wartość $-v$. Negując nierówność otrzymujemy:

$$-\beta \leq -v \leq -\alpha$$

Słownie:

Dolny zakres jednego gracza zamienia się na górny zakres drugiego. Oprócz zwykłej zamiany wartości, z racji tego że mamy do czynienia z grą o sumie zerowej, ta sama wartość znaczy dla przeciwnika coś odwrotnego - dlatego negujemy

Finalnie implementacja w scali jest dosyć bardziej złożona (nie możemy sobie od tak skończyć w potowie funkcji `foldleft`). Postępujemy się tutaj klasycznymi sztuczkami rekurencyjnymi

```
1 def negamax(s: S, depth: Int, alpha: Double, beta: Double): Double =
2   visited += 1
3   if depth <= 0 || s.isGameOver then heuristic(s)
4   else {
5     def findMaxForState(states: List[S], currentAlpha: Double): Double =
6       =
7       states match {
8         case Nil => currentAlpha
9         case head :: tail =>
10            val score = -1 * negamax(head, depth - 1, -beta, -
11              currentAlpha)
12
13            val newAlpha = math.max(currentAlpha, score)
14
15            if newAlpha >= beta then newAlpha
16            else findMaxForState(tail, newAlpha)
17          }
18     findMaxForState(s.generateStates.toList, alpha)
19   }
```

1.7 Ocena heurestyk

Do oceny heurestyk przygotowałem tryb pracy programu `tournament`, który pozwala na przetestowanie każdej z heurestyk w trybie “każdy z każdym”, na każdym poziomie głębokości, wyznaczonym z podanego zakresu. Domyślny zakres głębokości wykorzystywany przez program to 3 do 5, dlatego dla 3 heurestyk mamy łącznie $3 \cdot 3 \cdot 3 = 27$ różnych możliwości.

1.7.1 Średnia liczba rund dla każdej z heurystyk

Heurystyka	Średnia liczba rund
balanced	72
Progressive	61
Simple	55

Jak widzimy na powyższym wykresie, heurystyki najbardziej skrajne, zajmowały najmniejszą liczbę rund. Takich wyników można się było spodziewać. Heurystyka grającą bezpiecznie, nie szarżującą, i dosyć obronna, będzie utrzymywać się najdłużej bez wygranej i przegranej. To wynika z zachowawczej charakterystyki gry

Heurystyka progresywna jak i najprostsza, licząca pionki zajmuje najmniejszą liczbę rund. Nie analizując innych wyników mogą nam się nasunąć wnioski, takie że agresywna heurystyka bardzo szybko dąży do rozwiązania dlatego gra szybko się kończy (niekoniecznie wygraną), natomiast najprostsza heurystyka może być po prostu za słaba, nie dążyć za szybko do zwycięstwa i przez to jej gry mogą się częściej kończyć (szybką) porażką

1.7.2 Liczba odwiedzonych węzłów

Głębokość	balanced	progressive	simple	Sum
3	123,08%	120,72%	56,20%	100%
4	115,83%	93,68%	90,49%	100%
5	157,92%	95,16%	46,93%	100%
Suma	152,35%	101,11%	52,46%	100%

1.7.3 Średnia wartość zwycięstwa

heurystyka 1	heurystyka 2	balanced	progressive	simple	Sum
balanced		1	1	1,33	1,11
progressive		1,66	1	1,33	1,33
simple		1,66	1,33	2	1,66
Sum		1,44	1,11	1,55	-

Wartość powyższej oznacza:

- jeżeli wartość jest bliższa 1, oznacza to, że heurystyka 1 wygrywała częściej
- jeżeli wartość jest bliższa 2, oznacza to, że heurystyka 2 wygrywała częściej

Analizując powyższą tablicę, można wyciągnąć konkretne wnioski co do skuteczności poszczególnych algorytmów. Na prowadzenie wysuwa się heurystyka “balanced”, która jako gracz 1 osiąga najniższą średnią (1,11), co oznacza niemal same wygrane. Jest ona również najtrudniejszym przeciwnikiem dla pozostałych, gdy występuje w roli gracza 2 (średnia w kolumnie 1,44).

Z kolei heurystyka “simple” jako gracz 1 wypada obecnie najstabilniej, osiągając średnią 1,66. Bardzo wyraźnie widać w jej przypadku specyficzny trend w starciu z samą sobą – wynik 2 wskazuje na pełną wygraną gracza 2. Potwierdza to tezę, że przy uproszczonej logice premiowana jest strategia defensywna, gdzie gracz wykonujący ruch jako drugi ma ostatnie słowo w wymianach.

Źródła:

- <https://github.com/lagodiuk/tic-tac-toe-minimax-scala>
- http://pbeling.w8.pl/game-theory/praktyczne_aspekty_programowania_gier_logicznych.pdf
- <https://www.youtube.com/watch?v=l-hh51ncgDI>